JAVA FULL STACK DEVELOPER VIRTUAL INTERNSHIP

An Internship Report submitted
in partial fulfillment of the
Requirements for the award of the degree of

Bachelor of Technology

In

Artificial Intelligence & Machine Learning

By

SHAIK SANA

Reg No: 22H71A6151

OFFERED BY

AICTE-EDUSKILLS

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

DVR & Dr. HS

MIC College of Technology

Autonomous

Kanchikacherla– 521180, NTR Dist, Andhra Pradesh

JAN - MAR 2026

DVR & Dr. HS
MIC College of Technology
ISO 9001:2015 Certified Institute
(Approved by AICTE & Permanently Affiliated to JNTUK, Kakinada)
Kanchikacherla - 521180, NTR Dist., A.P., India
Phones : 08678 - 273535 / 94914 57799 / 73826 16824
E mail: office@mictech.ac.in, Website: www.mictech.edu.in

AUTONOMOUS
Accredited by NAAC With Grade A+

CERTIFICATE

This is to certify that the Internship Report entitled "***JAVA FULL STACK DEVELOPER Virtual Internship***" submitted by ***SHAIK SANA (22H71A6151),*** to the DVR & Dr. HS MIC College of Technology in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in ***Artificial Intelligence & Machine Learning*** is a Bonafide record of work.

Internship Coordinator ***Head of the Department***

Examiner1 ***Examiner2***

Principal

NATIONAL INTERNSHIP PORTAL

अखिल भारतीय तकनीकी शिक्षा परिषद्
All India Council for Technical Education

EduSkills®
Nation Building Through Skills

AICTE - EduSkills 2025 VIRTUAL INTERNSHIP

Certificate of Virtual Internship

This is to certify that

shaik sana

DVR & Dr. HS MIC College of Technology

has successfully completed 10 weeks

Android Developer Virtual Internship

During July - September 2025

We wish him / her all the best for the future endeavours

Supported By : **India Edu Program**

Google for Developers

Karthik Padmanabhan Developer Ecosystem Lead MENA & India, Google	**Shri Buddha Chandrasekhar** Chief Coordinating Officer (CCO) NEAT Cell, AICTE	**Dr. Satya Ranjan Biswal** Chief Technology Officer (CTO) EduSkills

Certificate ID :8cfd9f603453c28498bc814cbf7c4de3
Student ID :STU6625388b303711713715339

INTERNSHIP GRADE A

GRADE- O (Outstanding):90-100 | E (Excellent):80-89 | A (Very Good):70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose constant guidance and engagement crown all the efforts with success. I thank our college management and respected ***Sri D. PANDURANGA RAO, CEO*** for providing us the
necessary infrastructure to carry out the Internship.

I express my sincere thanks to ***Dr. T. Vamsee Kiran, Principal*** who has been a great source of inspiration and motivation for the internship program.

I profoundly thank ***Dr. G Sai Chaitanya Kumar, Head, Department of AI*** for permitting me to carry out the internship.

I am thankful to the ***AICTE and Android Developer*** for enabling me an opportunity to carry out the internship in such a prestigious organization.

I am thankful to our Internship Coordinator ***Dr. A. Kalyan Kumar,*** Associate Professor, Department of AI for their internal support and professionalism who helped us in completing the internship on time.

I take this opportunity to express our thanks to one and all who directly or indirectly helped me in bringing this effort to present form.

Finally, my special thanks go to my family for their continuous support and help throughout and for their continual support and encouragement for the completion of the Internship on time.

ABSTRACT

In the rapidly evolving landscape of mobile technology, Android stands as a dominant force, powering millions of devices worldwide. This project aims to leverage the Android platform to develop a cutting-edge mobile application focused on enhancing user experience and addressing specific needs within a targeted user base.

The primary objectives of this Android development project include User-Centric Design, Functionality and Features, Performance Optimization, Security and Privacy, Cross-Platform Compatibility, Testing and Quality Assurance.

The successful completion of this Android development project is expected to result in a feature-rich, user-friendly mobile application that not only meets the specified requirements but also establishes a foundation for future iterations and improvements. This project aligns with the broader goal of leveraging technology to enhance the overall user experience in the mobile domain.

ORGANIZATION INFORMATION:

Edu Skills and AICTE have launched a Virtual Internship program on Android Developer, which is sponsored by Google, with the goal of creating an industry-ready workforce that will eventually become leaders in emerging technologies. Google provides higher education institutions with a free, ready-to-teach the curriculum that prepares students to pursue industry-recognized certifications and in-demand developer jobs. Our curriculum helps educators stay at the forefront of Android Developer innovation so that they can equip students with the skills they need to get hired in one of the fastest growing industries.

Google LLC (is an American multinational technology company focusing on artificial intelligence, online advertising, search engine technology, cloud computing, computer software, quantum computing, e-commerce, and consumer electronics. It has been referred to as "the most powerful company in the world" and as one of the world's most valuable brands due to its market dominance, data collection, and technological advantages in the field of artificial intelligence. Alongside Amazon, Apple Inc., Meta, and Microsoft, Google's parent company Alphabet Inc. is one of the five Big Tech companies.

INDEX

YOUR FIRST ANDROID APP

This is the recommended course to start learning Android! Build a series of apps using Jetpack Compose, the modern toolkit for creating beautiful user interfaces on Android. You will write these apps in the Kotlin programming language and learn best practices in MaterialDesign, app architecture, data storage, fetching data from the network, testing, and more. This program consists of live sessions, Hands-on practical activities, mentoring support and working on super badges on Trailhead platform. In order to help all beginners, understand the salesforce ecosystem and its products, we have curated a few best modules on the trailhead platform that will help you to get ready for the Bootcamp.

What is an Android Developer?

To become an Android developer, one typically needs a strong understanding of programming concepts, experience with Java or Kotlin, and familiarity with Android development tools. Many Android developers have a background in computer science or a related field, although practical experience and self-learning are also common paths into the profession. Continuous learning is crucial in this dynamic field to keep up with the evolving Android ecosystem.

What Does an Android Developer do?

The Android developer does the application of any software in android and ios software applications etc. To become an Android developer, one typically needs a strong understanding of programming concepts, experience with Java or Kotlin, and familiarity with Android development tools. You use a tool called a code editor to write and edit your code. It's similar to a text editor in which you can write and edit text, but a code editor also provides functionality to help you write code more accurately. For example, a code editor shows autocomplete suggestions while you type, and displays error messages when code is incorrect. To practic the basics of the Kotlin language, you will use an interactive code editor calledthe Kotlin Playground. You can access it from a web browser, so you don't need to install anysoftware on your computer. You can edit and run Kotlin code directly in the Kotlin Playground and see the output.

Note: That you can't build Android apps within the Kotlin Playground. In later pathways, you will install and use a tool called Android Studio to write and edit your Android app code.

Part of a function:

Here'san analogy.You write out step-by-step instructions on how to bake a chocolate cake. This set of instructions can have a name: bake Chocolate Cake. Every time you want to bake a cake, you can execute the bake Chocolate Cake instructions. If you want 3 cakes, you need to execute the bake Chocolate Cake instructions 3 times. The first step is defining the steps and giving it a name, which is considered defining the function. Then you can refer to the steps anytime you want them to be executed, which is considered calling the function.

Kotlin style guide:

Throughout this course, you'll learn about good coding practices to follow as an Android developer. One such practice is to follow Google's Android coding standards for code written in Kotlin. The complete guide is called a style guide and explains how code should be formatted in terms of visual appearance and the conventions to follow when writing code.

1.2 Set up android studio:

Androidandinstallandroidstudio:

- AndroidStudio system requirements:
- The following are the Android Studio system requirements for Linux.
- Any 64-bit Linux distribution that supports Gnome, KDE, or Unity DE; GNU C Library 2.31 or later.
- x86_64 CPU architecture; 2nd generation Intel Core or newer, or AMD processor with support for AMD Virtualization (AMD-V) and SSSE3
- 8 GB RAM or more
- 8 GB of available disk space minimum (IDE + Android SDK + Android Emulator)
- 1280 x 800 minimum screen resolution.

Mac OS: verify system requirement:

- Android Studio system requirements(macOS)
- The following are the system requirements for Android Studio on macOS.
- MacOS® 10.14 (Mojave) or higher
- ARM-based chips, or 2nd generation Intel Core or newer with support for Hypervisor Framework
- 8 GB RAM or more
- 8 GB of available disk space minimum (IDE + Android SDK + Android Emulator)
- 1280 x 800 minimum screen resolution.

1.3 Build a Basic layout:

Create a low-fidelity prototype:

When you begin a project, it's useful to visualize how UI elements need to fit together on the screen. In professional-development work, oftentimes there are designers or design teams that provide developers with UI mockups, or designs, that contain exact specifications. However, if you don't work with a designer, you can create a low-fidelity, or low-fi, prototype on your own. Low-fi prototype refers to a simple model, or drawing, that provides a basic idea of what the
app looks like.

Spacing and alignment:

You can use Modifier arguments, such as padding and weight modifiers, to help with the arrangement of composable.

You can also use Spacer composable to make spacing more explicit.

Color customization.

You can use custom color with the Color class and the color hex code (a hexadecimal way to represent a color in RGB format). For example, the green color of Android has a hex code of #3DDC84. You can make your text the same green color with this code:

Text ("Example", COLOR = COLOR(0xFF3ddc84))

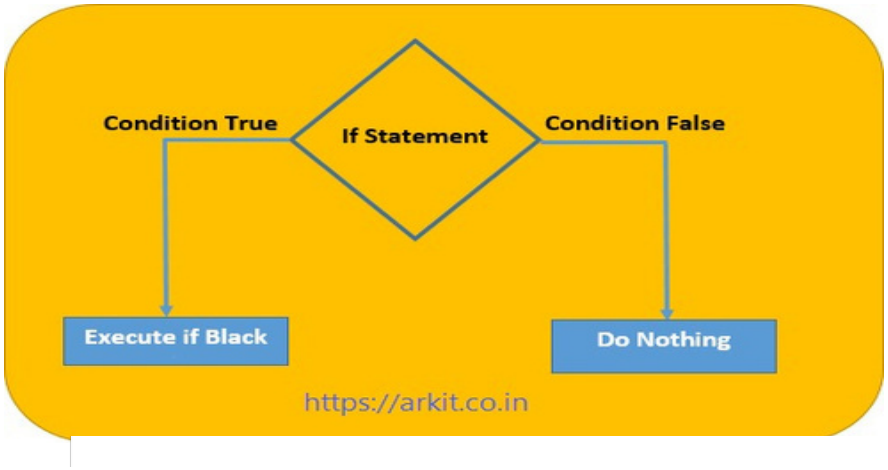
BUILDING APP UI

2.1 KOTLIN FUNDAMENTALS:

Using if/else statements to express conditions:

In life, it's common to do things differently based on the situation that you face. For example, if the weather is cold, you wear a jacket, whereas if the weather is warm, you don't wear a jacket. Decision-making is also a fundamental concept in programming. You write instructionsabout how a program should behave in a given situation so that it can act or react accordingly when the situation occurs. In Kotlin, when you want your program to perform differentactions based on a condition, you can use an if/else statement. In the next section, you writean ifstatement .

Write if conditions with Boolean expressions:



When statement for multiple branches:

Your traffic Light Color program looks more complex with multiple conditions, also known as branching. You may wonder whether you can simplify a program with an even larger number of branches.

In Kotlin, when you deal with multiple branches, you can use the when statement instead of the if/else statement because it improves readability, which refers to how easy it is for human readers, typically developers, to read the code. It's very important to consider readability when you write your code because it's likely that other developers need to review and modify your code throughout its lifetime.

2.2 A Button to an app:

Build the roll-dice logic:

Take the button interactive In the Dice With Button And Image() function before the Column() function, create a result variable and set it equal to a 1 value.

Take a look at the Button composable. You will notice that it is being passed an on Click parameter which is set to a pair of curly braces with the comment /*TODO*/ inside the braces. The braces, in this case, represent what is known as a lambda, the area inside of the braces being the lambda body. When a function is passed as an argument, it can also be referred to as a " callback".

Main Activity. Kt

Button (on Click = { /*TODO*/ })

A lambda is a function literal, which is a function like any other, but instead of being declared separately with the fun keyword, it is written inline and passed as an expression. The Button composable is expecting a function to be passed as the onClick parameter. This is the perfect place to use a lambda, and you will be writing the lambda body in this section.

In the Button () function, remove the /*TODO*/ comment from the value of the lambda body of the on Click parameter.

A dice roll is random. To reflect that in code, you need to use the correct syntax to generate a random number. In Kotlin, you can use the random () method on a number range. In the on Click lambda body, set the result variable to a range between 1 to 6 and then call the random () method on that range. Remember that, in Kotlin, ranges are designated by two periods between the first number in the range and the last number in the range.

2.3 INTERACT WITH UI AND STATES:

Build a Static UI with composable:

Create a low-fidelity prototype

Low-fidelity (low-fi) prototype refers to a simple model, or drawing, that provides a basic idea of what the app looks like.
Create a low-fi prototype:

Think about what you want to display in your Art Space app and who the target audience is.

On your preferred medium, add elements that make up your app. Some elements to consider

include:

Artwork image

Information about the artwork, such as its title, artist, and year of publication

Any other elements, such as buttons that make the app inter active and dynamic.

Put these elements in different positions and then evaluate them visually. Don't worry about getting it perfect the first time. You can always settle on one design now and iteratively improve later.

Built for different screen sizes:

One of the strengths of Android is that it supports many devices and screen sizes, which means that the app that you build can reach a wide range of audiences and be used in a multitude of ways. To ensure the best experience for all users, you should test your apps on devices that your app intends to support. For example, in the current sample app, you may have initially designed, built, and tested the app for mobile devices in portrait mode. However, some of your
users might find your app enjoyable to use on a larger screen in landscape mode.

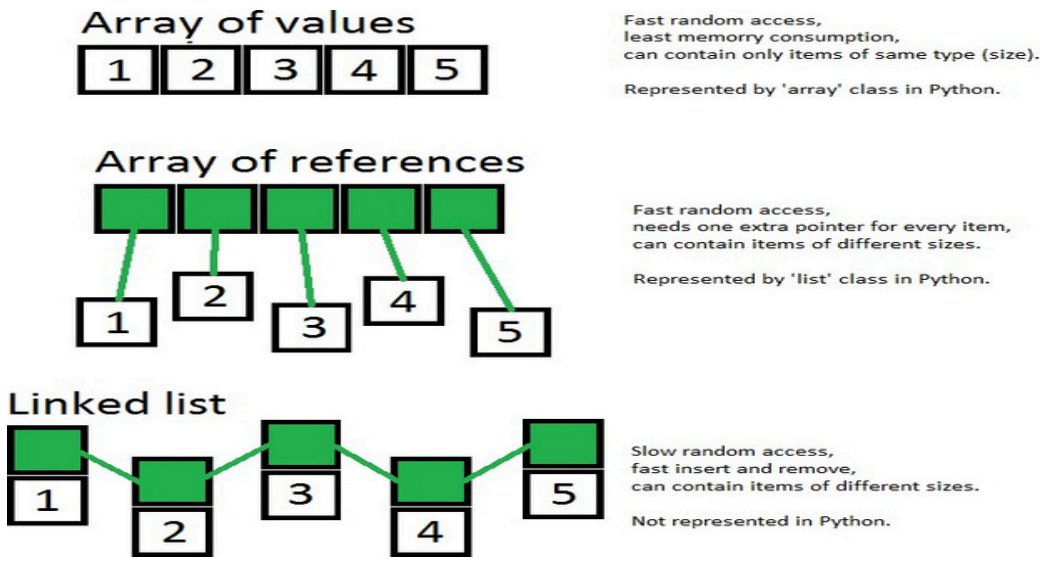
Display List and Use Material Design

3.1 More Kotlin fundamentals:

What is an array?

An array is the simplest way to group an arbitrary number of values in your programs.

Like a grouping of solar panels is called a solar array, or how learning Kotlin opens up an array of possibilities for your programming career, an Array represents more than one value. Specifically, an array is a sequence of values that all have the same data type.



LIST:

A listis an ordered, resizable collection, typically implemented as a resizable array. When the array is filled to capacity and you try to insert a new element, the array is copied to a new bigger array.

Map Collection:

AMap is a collection consisting of keys and values. It's called a map because unique keys are mapped to other values. A key and its accompanying value are often called a key-value pair.

3.2 Build a scrollable list:

Launcher icons:

Thegoalisfor your launcher icon to look crisp and clear, regardless of the device model or screen density. Screen density refers to how many pixels per inch or dots per inch (dpi) are on the screen. For a medium-density device, there are 160 dots per inch on the screen, while an extra-extra-extra-high-density device has 640 dots per inch on the screen.

Change APP Icon:

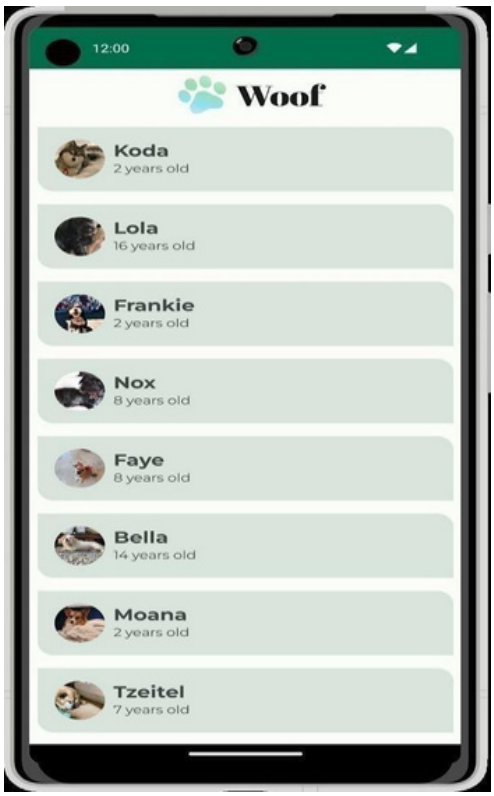
Gobackto Android Studio to use the new assets you just downloaded.
First, delete the old drawable resources that contain the Android icon and green grid background. In the Project view, right-click on the file and choose Delete.

DELETE:
drawable/ic_launcher_background.xml
drawable/ic_launcher_foreground.xml

3.3 Build Beautiful apps:

App Overview:

In this code lab, you create Woof, an app that displays a list of dogs and uses Material Design to create a beautiful app experience.



Through this code lab, we will show you some of what is possible using Material Theming. Use this code lab for ideas of how to use Material Theming to improve the look and feel of the

apps you create in the future.

COLOR PALETTE:

Below are the palettes for both light and dark themes that we will create.

Here is the final app in both light theme and dark theme.

Theme file:

The Theme. kt file is the file that holds all the information about the theme of the app, which is defined through, typography, and shape. This is an important file for you to know. Inside of the file is the composable Woof Theme (), which sets the, typography, and shapes of the app.

Navigation and app architecture

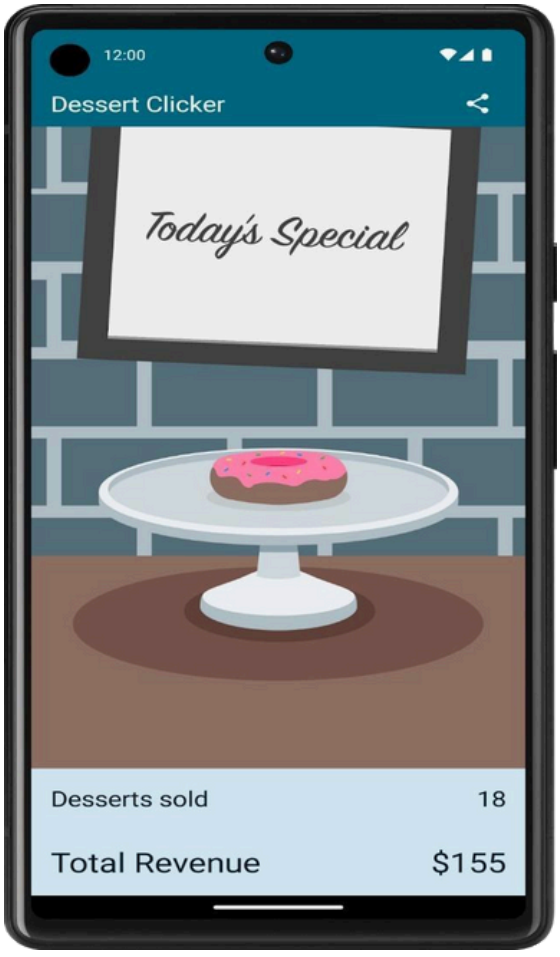
4.1 Architecture Components:

Stages of the Active Life cycle:

App overview:

Inthiscode lab, you work with a starter app called Dessert Clicker. In Dessert Clicker, each time the user taps a dessert on the screen, the app "purchases" the dessert for the user. The app updates values in the layout for the:

Number of desserts that are "purchased"
Total revenue for the "purchased" desserts



This app contains several bugs related to the Android lifecycle. For example, in certain circumstances, the app resets the dessert values to 0.

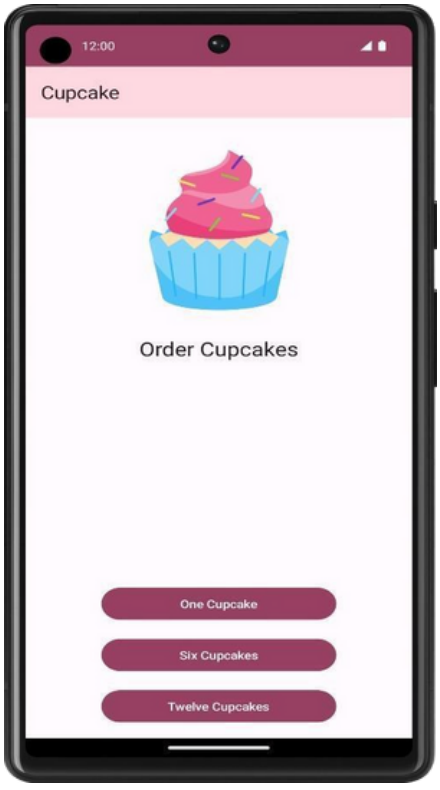
4.2Navigation in Jetpack Compose:

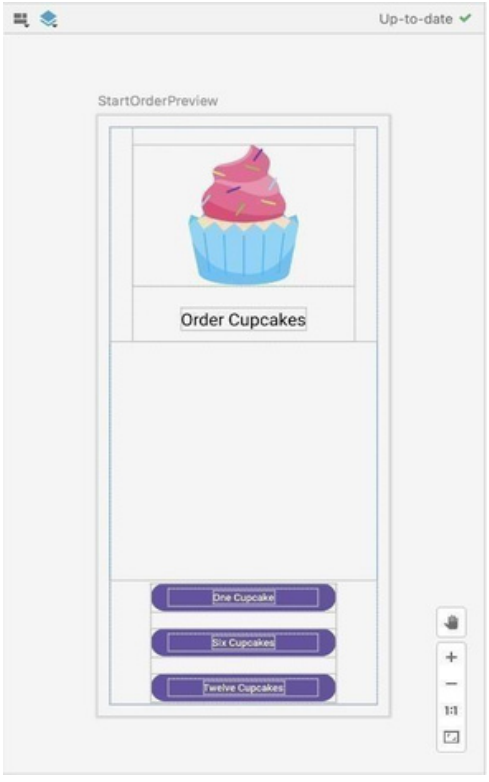
Navigate between screen with compose: App walk though:

TheCupcakeapp is a bit differentthan the apps you've worked with so far. Instead of all the content displaying on a single screen, the app has four separate screens, and the user can navigate through each screen while ordering cupcakes. If you run the app, you won't see anything and you won't be able to navigate between these screens since the navigation component is not added to the app code yet. However, you can still check the composable previews for each screen and match them with the final app screens below.

Start order screen:

Thefirstscreenpresents the user with three buttons that correspond to the quantity of cupcakes to order.

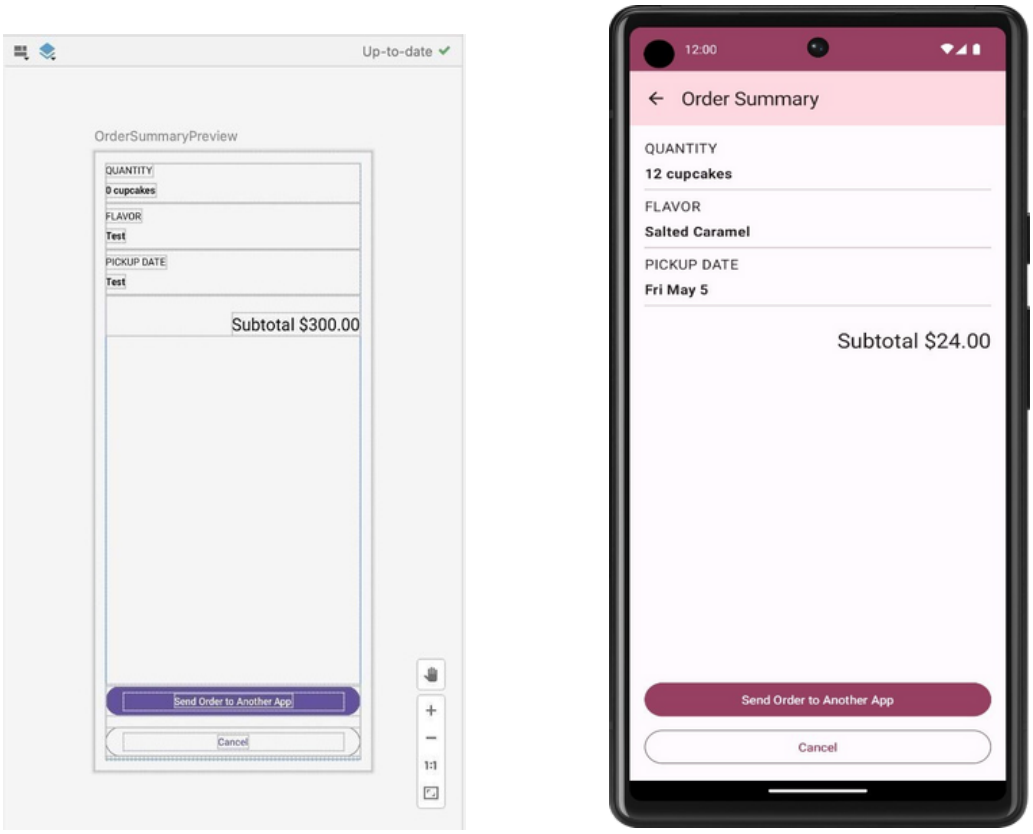




Order Summary screen:

After selecting the pickup date, the app displays the Order Summary screen

where the user can review and complete the order.



This screen is implemented by the Order Summary Screen composable in Summary Screen. kt.

The layout consists of a Column containing all the information about their order, a Text composable for the subtotal, and buttons to either send the order to another app or cancel the order and return to the first screen.

If users choose to send the order to another app, the Cupcake app displays an Android Share Sheet that shows different sharing options.

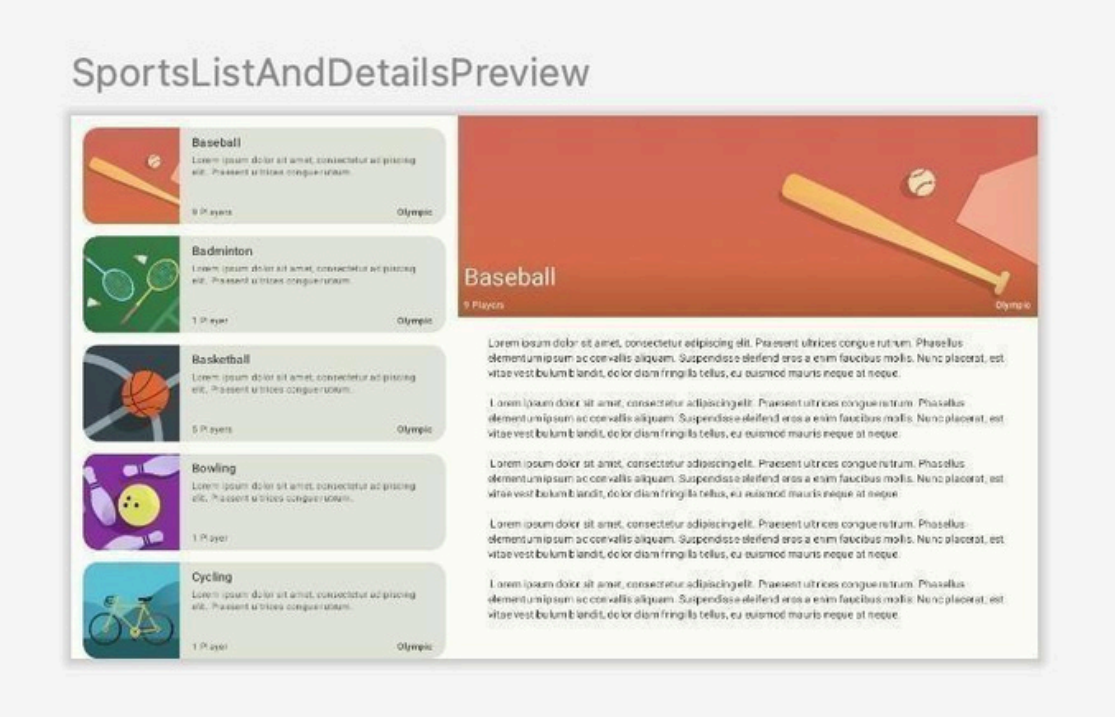
4.3 Adapt for different screen sizes:

Create the expanded screen in code:

Now that you have a clear view of how the expanded layout looks, let's translate that into code.

Create a composable for the expanded screen that shows both the list and the details screen.
You can name it Sports List And Details and place it in the Sports Screens. kt file.

Create a composable preview to simplify verification of the UI for the Sports List And Details composable.



Ensure that the back button behavior is appropriate for the expanded screen. When the user presses the back button, you want them to exit the app when the main screen appears. You can change this behavior by passing the appropriate lambda to the Sports Details composable.
Hint: you can have access to the app's Activity from (Local, current as Activity).

Connect to the internet

5.1 Get Data from the internet:

Get data from the internet:

Youwork withthe appnamedMars Photos, which shows images of the Mars surface. This app connects to a web service to retrieve and display Mars photos. The images are real-life photos from Mars, captured from NASA's Mars rovers. The following image is a screenshot of the final app, which contains a grid of images.

The version of the app you build in this code lab won't have a lot of visual flash. This code lab focuses on the data layer part of the app to connect to the internet and download the raw property data using a web service. To ensure that the app correctly retrieves and parses this data, you can print the number of photos received from the backend server in a Text composable..

JSON parsing:

The response from a web service is often formatted in JSON, a common format to represent structured data.

A JSON object is a collection of key-value pairs.

5.2 Load and display images from the internet:

To practice the concepts you learned in this unit, including coroutines,

Retrofit, and you're going to build an app on your own that displays a list of books with images from the Google Books API.

The app is expected to do the following:

Make a request to the Google Books API using Retrofit.

Parse the response.

Display asynchronously downloaded images of the books along with their titles in a vertical grid.

Implement best practices, separating the UI and data layer, by using a repository.

Write tests for code that requires the network service, using dependency injection.

Data persistence

6.1 Introduction to SQL:

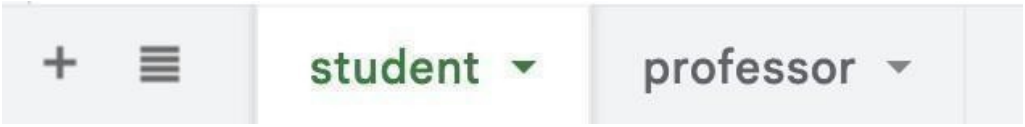
Use to read and write to a database:

Key concept of relation database:

What is a database?

Ifyou arefamiliarwith a spreadsheet program like Google Sheets, you are already familiar with a basic analogy for a database.

A spreadsheet consists of separate data tables, or individual spreadsheets in the same workbook.



Representing data with SQLite:

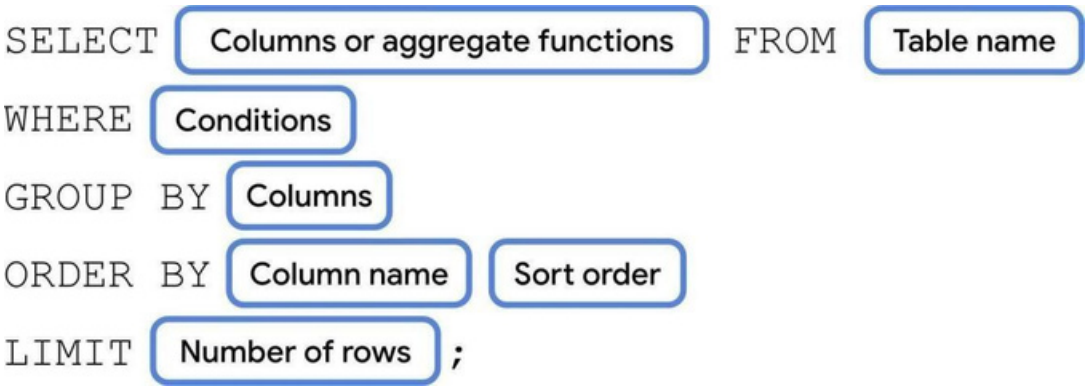
InKotlin, you're familiar with data types like Int and Boolean. SQLite databases use data types too! Data table columns must have a specific data type. The following table maps common Kotlin data types to their SQLite equivalents.

Summary:

Adatabaseusing SELECT statements, including WHERE, GROUP BY, ORDER BY, and LIMIT clauses to filter results. You also learned about frequently used aggregate functions, the DISTINCT keyword to specify unique results, and the LIKE keyword to perform a text search on the values in a column. Finally, you learned how to INSERT, UPDATE, and DELETE rows in a data table.

These skills will translate directly to Room, and with your knowledge of SQL, you'll be more than prepared to take on data persistence in your future apps.

SELECT statement syntax:



6.2 Use Room for data persistence:

Main components of a Room:

While it is easy to work with in-memory data using data classes, when it comes to persisting data, you need to convert this data into a format compatible with database storage. To do so, you need tables to store the data and queries to access and modify the data.

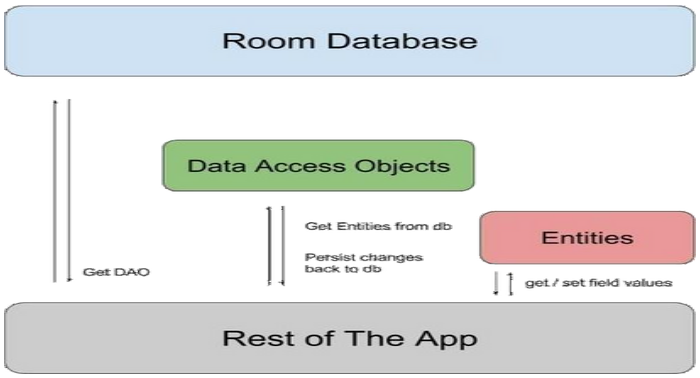
The following three components of Room make these workflows seamless.

Room entities represent tables in your app's database. You use them to update the data stored in rows in tables and to create new rows for insertion.

Room DAOs provide methods that your app uses to retrieve, update, insert, and delete data in the database.

Room Database class is the database class that provides your app with instances of the DAOs associated with that database.

You implement and learn more about these components later in the code lab. The following diagram demonstrates how the components of Room work together to interact with the database.



Create the Database:

In the data package, create a Kotlin class Inventory Database. kt.

In the Inventory Database. kt file, make Inventory Database class an abstract class that extends Room Database.

Annotate the class with @Database. Disregard the missing parameters error, which you fix in the next step.

6.3 Store and access datausing keys with Data Store:

Create a Flight search App:

Relational databases and Structured Query Language (SQL), integrated a relational database into an app with Room, and learned about Preferences Data Store for persisting settings and UI state. It's time to put everything you've learned into practice.

In this project, you'll build the Flight Search app in which users enter an airport and can view a list of destinations using that airport as a departure. This project gives you the opportunity to practice your skills with SQL, Room, and Data Store by offering you a set of app requirements that you must full fill. In particular, you need the Flight Search app to meet the following requirements:

•Provide a text field for the user to enter an airport name or International Air Transport Association (IATA) airport identifier.

•Query the database to provide autocomplete suggestions as the user types.

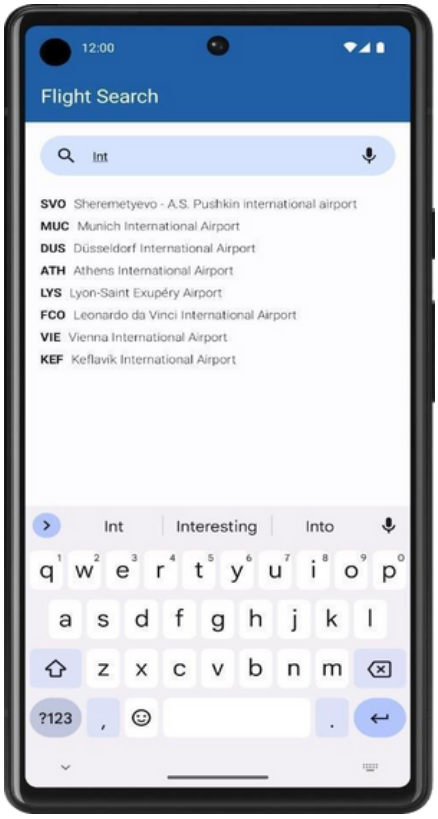
•When the user chooses a suggestion, generate a list of available flights from that airport, including the IATA identifier and airport name to other airports in the database.

•Let the user save fav individual routes.

•When no search query is entered, display all the user-selected routes in a list.

•Save the search text with Preferences Data Store. When the user reopens the app, the search text, if any, needs to prepopulate the text field with appropriate results from the database.

Plan your UI:



You're welcome to design your app however you like. As a guide, the following descriptions.

When the user clears the search box or does not enter a search query, the app displays a list of saved fav destinations, if any exist.

Work Manager

Schedule tasks with Work Manager:

Ensure unique work:

Now that you know how to chain workers, it's time to tackle another powerful feature of Work Manager: unique work sequences.

Sometimes, you only want one chain of work to run at a time. For example, perhaps you have a work chain that syncs your local data with the server. You probably want the first data sync to complete before starting a new one. To do this, you use begin Unique Work() instead of begin With(), and you provide a unique String name. This input names the entire chain of work requests so that you can refer to and query them together.

You also need to pass in an Existing Work Policy object. This object tells the Android OS what happens if the work already exists. Possible Existing Work Policy values are REPLACE, KEEP, APPEND, or APPEND_OR_REPLACE.

In this app, you want to use REPLACE because if a user decides to blur another image before the current one finishes, you want to stop the current one and start blurring the new image.

You also want to ensure that if a user clicks Start when a work request is already enqueued, then the app replaces the previous work request with the new request. It does not make sense to continue working on the previous request because the app replaces it with the new request anyway.

In the data/Work Manager Repository. kt file, inside the apply Blur() method, complete the following steps:

Remove the call to begin With() function and add a call to the begin Unique Work() function.

For the first parameter to the begin Unique Work() function, pass in the constant IMAGE_MANIPULATION_WORK_NAME.

For the second parameter, the existing Work Policy parameter, pass in Existing Work Policy. REPLACE.

For the third parameter, create a new One Time Work Request for the Cleanup Worker.

Compose with Views

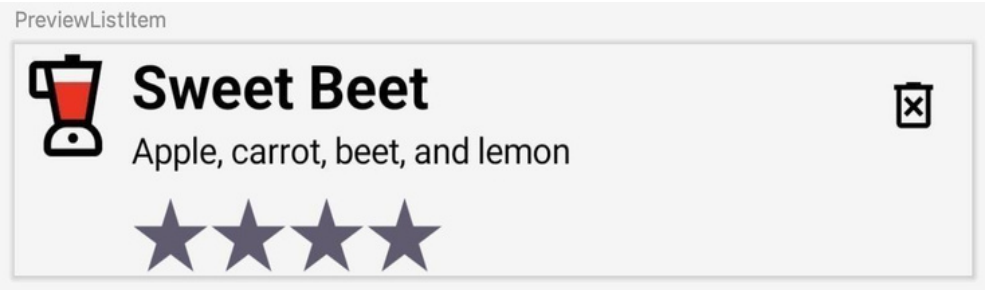
8.1 Android Views and Compose in Views:

This code lab uses the Juice Tracker app solution code from the Build an Android App with Views as the starter code. The starter app already saves data using the Room persistence library. The user can add juice info to the app database, like juice name, description, color, and rating.

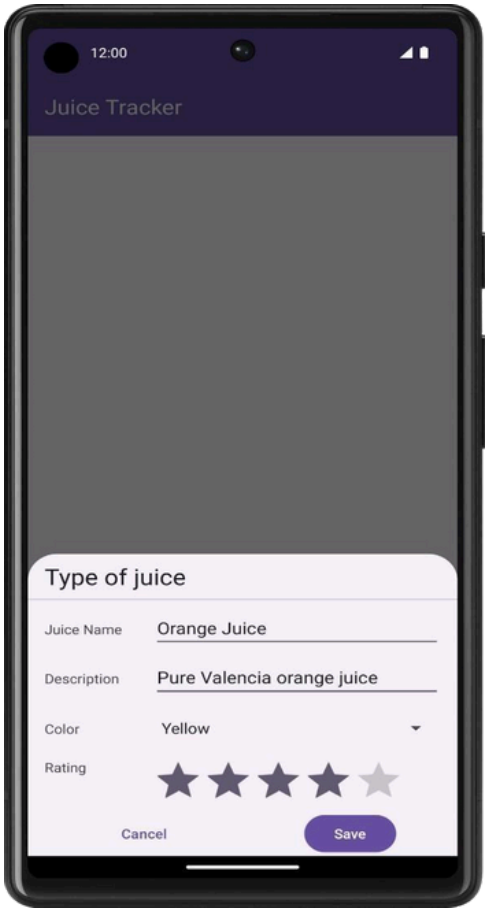
In this code lab, you convert the view-based list item to Compose.

Implement list items functions:

Writeapreviewfunctiontopreviewthe list items composable.



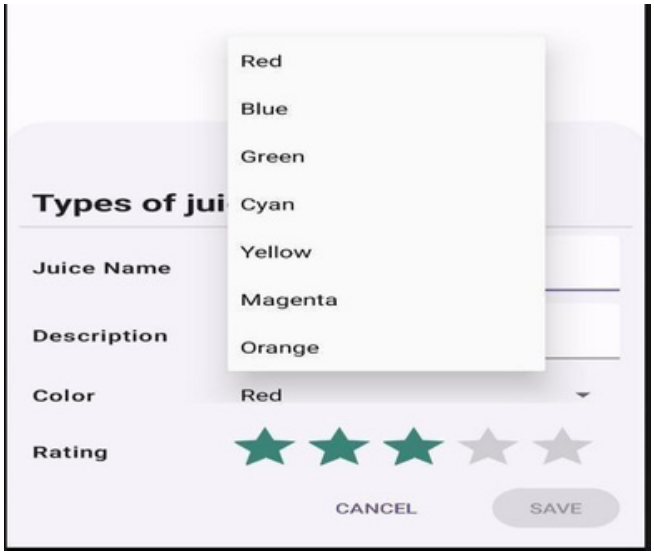
Run the app. Add your favorite juice. Notice the shiny compose list item.



8.2 Views in Compose: Gradle configuration:

Complete the enter dialog: In this section, you complete the entry dialog by creating the color spinner and the rating bar.

The color spinner is the component that lets you choose a color, and the rating bar lets you select a rating for the juice. See the design below:



Create the color spinner:

Toimplement aspinner in Compose, the Spinner class must be used. Spinner is a View component, as opposed to a Composable, so it must be implemented using an interop. In the bottom sheet directory, create a new file called Color Spinner Row. kt.

Create a new class inside the file called Spinner Adapter.

In the constructor for the Spinner Adapter, define a callback parameter called on Color Change

that takes an Int parameter. The Spinner Adapter handles the callback functions for the Spinner.

Bottom sheet/Color Spinner Row. Kt

```
class Spinner Adapter (Val on Color Change: (Int) -> Unit) {
}
```

The Android View Composable takes three parameters:

The factory lambda, which is a function that creates the View.

The update callback, which is called when the View created in the factory is inflated.

A Composable modifier.

CONCLUSION

The conclusion for an Android developer is that staying updated on the latest technologies, frameworks, and design principles is crucial for success. Continuous learning, effective problem-solving, and a keen understanding of user experience contribute to building successful and innovative Android applications. Additionally, collaborating with the vibrant Android developer community can provide valuable insights and support for professional growth.

As I conclude my Internship, This Comprehensive report outlines my key accomplishments, skills acquired and experiences gained during my tenure. I appreciate the opportunities provided and I am confident that the skills and expertise gained will significantly enhance my professional endeavors.